



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #9
Pre-Processor Directives***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Pre-processor Directives**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

The execution of a C program follows a multi-stage process that includes: Preprocessing, Compilation, Linking, Loading, and Execution. The first stage, preprocessing, is carried out by the C preprocessor. Preprocessor directives in C are instructions that are **executed before the actual compilation of the program begins**. Each directive starts with a **# symbol** and these directives are **commonly used to include header files, define constants and macros, and control conditional compilation**. You can observe the effect of the preprocessor on source files by using the **-E option with the gcc compiler**. Although it plays a crucial role in preparing the code, it operates independently from the compiler.

Key Types of Preprocessor Directives

- **Macro Definition**
- **File Inclusion**
- **Macro Undefinition**
- **Conditional Compilation**
- **Other Directives**

Macro Definition :

A macro is a fragment of code defined using the **#define** directive that gets replaced by its value or expression during the preprocessing stage. **#define** is used **to define constants or expressions that are replaced by their values before compilation**. Begins with the **#define** keyword and **does not require a semicolon** at the end.

Characteristics of a Macro:

1. **Textual Substitution/ Macros doesn't judge anything:** The preprocessor replaces all occurrences of the macro name with its defined value or expression before the actual compilation begins.
2. **Macros can have expression: No Data Type Enforcement:** Macros are **not type-checked**, unlike variables or functions. This can lead to unexpected behavior if not used carefully.

3. **No Memory Allocation:** Macros do not consume memory during program execution.
4. **Improves Code Readability:** Replacing magic numbers or repeated expressions with macros can make code easier to read and maintain.
5. **Parentheses are Crucial:** Always use parentheses around macro arguments and the entire macro body to avoid precedence errors during expansion.
6. **Can Take Arguments (Function-like Macros):**
7. **Cannot be debugged easily:** Since macros are replaced before compilation, they don't appear in the debug information, making them harder to trace.
8. **Can be Undefined:** Use `#undef` to remove a macro definition.

Advantages of Macros:

- **Faster Execution:** Macros are replaced directly in the code, so there's no function call overhead.
- **No Memory Usage for Storage:** Unlike variables, macros don't use memory at runtime since they are just text replacements.
- **Easier to Update Values:** If a constant value is used in many places, we can change it in one macro definition instead of everywhere.
- **Flexible and Reusable:** Function-like macros can be reused for common operations like squaring or finding the maximum.
- **Conditional Compilation:** Macros help write code that works on different systems or conditions by including or excluding parts of the code.

NOTE: Use `gcc -E filename.c` to view the output of preprocessing stage on the terminal.

Coding Example_1: Macros doesn't judge anything. Pure textual substitution

```
#include<stdio.h>
#define PI 3.14           // symbolic constant, macro, plain text substitution
// It doesn't judge anything
// No memory is allocated for this
int main()
{
    printf("%f",PI); return 0;
}
```

```
int main()
{
    printf("%f",3.14); return 0;
}
```

Coding Example_2: No memory allocated for macros. Addressing access is a Compile-time error

```
#define MAX 10
int main()
{
    int a[MAX];
    scanf("%d", &MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    int a[10];
    scanf("%d", &10);
    return 0;
}
```

Compilation Error:

```
check.c:5:13: error: lvalue required as unary '&' operand
    scanf("%d", &MAX);
                  ^
```

Coding Example_3: Defining a string using macro

```
#define NAME "Admin - Sindhu R Pai"
int main()
{
    printf("Welcome %s", NAME);
}
// Replace %s with %d and see the output.
```

Preprocessing stage output:

```
int main()
{
    printf("Welcome %s", "Admin - Sindhu R Pai");
}
```

Final output:

```
C:\Users\Dell>a
Welcome Admin - Sindhu R Pai
```

Coding Example_4: Macro having an expression

```
#include<stdio.h>
#define EXPR 2+5*1
int main()
{
    printf("Value is %d\n",EXPR);
    //STR replaced with 2+5*1 in
    //preprocessing stage.
    //During execution, expression is evaluated
    return 0;
}
```

Preprocessing stage output:

```
int main()
{
    printf("Value is %d\n",2+5*1);

    return 0;
}
```

Final output:

```
C:\Users\Dell>a
Value is 7
```

Coding Example_5: Macro with parameters

```
#include<stdio.h>
#define SUM(a,b) a+b // SUM is not a function. It is a parameterized macro.
int main()
{
    int p,q;    printf("Enter two numbers:\n");
    scanf("%d%d",&p,&q);
    printf("The sum of two numbers is %d\n",SUM(p,q));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    int p,q;
    printf("Enter two numbers:\n");
    scanf("%d%d",&p,&q);
    printf("The sum of two numbers is %d\n",p+q);
    return 0;
}
```

Coding Example_6: Usage of parentheses is crucial

```
#include<stdio.h>
#define sqr(x) (x*x) //change this to (x)*(x).
int main()
{
    printf("%d",sqr(2+3)); // 11(2+3*2+3)
                        // Not (2+3)*(2+3)
    return 0;
}
```

Pre-processing stage output:

```
int main()
{ printf("%d", (2+3*2+3));
  return 0;
}
```

Think -> What changes to be done in the above code to perform $(2+3)*(2+3)$?

Coding Example_7: Usage of parentheses is crucial

```
#include<stdio.h>
#define sqr(x) (x)*(x)
{
    printf("%d",sqr(2+3));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{ printf("%d", (2+3)*(2+3));
  return 0;
}
```

Coding Example_8: Usage of one macro in another

```
#define sqr(x) x*x
#define cube(x) sqr(x)*x // using sqr in cube
int main()
{
    printf("The cube of 9 is d\n",cube(9));
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("The cube of 9 is d\n",9*9*9);
    return 0;
}
```

Coding Example_9: Redefining the macro using assignment operator results in Error

```
#define MAX 200
int main()
{
    printf("Before change %d\n", MAX)
    MAX=20;
    // Error because no memory allocated for MAX
    printf("After change %d",MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("Before change %d\n", 200)
    200=20;
    printf("After change %d",200);
    return 0;
}
```

Results in Compile time Error:

```
check.c: In function 'main':
check.c:5:5: error: lvalue required as left operand of assignment
    MAX=20;
    ^
```

Coding Example_10: Redefining a macro with #define is allowed. But not advisable.

```
#define MAX 200
int main()
{
    printf("Before change %d\n", MAX);
    #define MAX 20
    printf("After change %d",MAX);
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("Before change %d\n", 200);check.c:5:0: warning: "MAX" redefined
    #define MAX 20
check.c:1:0: note: this is the location of the previous definition
    #define MAX 200

    printf("After change %d",20);
    return 0;
}
```

Results in Warning at compiletime.

```
check.c:5:0: warning: "MAX" redefined
    #define MAX 20
check.c:1:0: note: this is the location of the previous definition
    #define MAX 200
```

Differences between a Macro and an Enumeration

Aspect	Macro (#define)	Enum (enum)
Definition	Preprocessor directive (text replacement)	Language-level data type
Type Safety	No type checking (just replaces text)	Type-safe (values have an integer type)
Scope	Global (no scope control)	Limited to scope where defined
Value Assignment	Must be manually assigned	Auto-incremented from 0 (or custom values) if value not spec
Readability & Maintenance	May reduce clarity when used excessively	More readable for related named constants

Similarity between macro and enum in C:

- **Both are used to define named constants:** Macros like `#define MAX 100` and enums like `enum { MAX = 100 };` assign meaningful names to fixed values, improving code readability.
- **The address operator (&) cannot be used on either:** Since macros are replaced by text and enums are not variables stored in memory, you cannot use the `&` operator to get their address.

Pre-defined Macros

Predefined macros in C are special identifiers that are automatically defined by the compiler. They provide useful information about the compilation environment such as **the current file name, line number, date, and time of preprocessing**. These macros begin with **double underscores (__)** and are mainly used for **debugging, logging, or conditional compilation**. You don't need to define them manually — they are available by default in every C program.

- ☐ **__FILE__**: Current file name
- ☐ **__LINE__**: Current line number
- ☐ **__DATE__**: Compilation date
- ☐ **__TIME__**: Compilation time
- ☐ **__STDC__**: Defined as 1 if the compiler follows ANSI C standard

Coding Example_11: Usage of Pre-defined macros

```
#include <stdio.h>
int main() {
    printf("File Name : %s\n", __FILE__);
    printf("Line Number : %d\n", __LINE__);
    printf("Date : %s\n", __DATE__);
    printf("Time : %s\n", __TIME__);
    printf("ANSI C? : %d\n", __STDC__);
    return 0;
}
```

Preprocessing stage output

```
:
# 3 "check.c"
int main() {
    printf("File Name : %s\n", "check.c");
    printf("Line Number : %d\n", 5);
    printf("Date : %s\n", "May 19 2025");
    printf("Time : %s\n", "15:50:46");
    printf("ANSI C? : %d\n", 1);
    return 0;
}
```

After executing the code:

```
C:\Users\Dell>a
File Name : check.c
Line Number : 5
Date : May 19 2025
Time : 15:46:08
ANSI C? : 1
```


Uses of Pre-defined Macros in C

- **Debugging:** Print the current file name, line number, or function to trace issues using `__FILE__`, `__LINE__`, and `__func__`.
- **Logging and Error Reporting:** Automatically log the date and time of preprocessing using `__DATE__` and `__TIME__`.
- **Conditional Compilation:** Use macros like `__STDC__` to check if the compiler follows the ANSI C standard and conditionally include code.
- **Code Documentation:** Embed build details in the output (like version/date/time) for tracking code versions or builds.
- **Cross-Platform Code Support:** Help write code that adapts to different environments or compiler standards using built-in conditions.

File Inclusion Directives: Used to include the contents of one file into another before compilation. This is done using the **#include** preprocessor directive.

- **Standard Library Files or Header files:** This contain the declarations of predefined functions like `printf()`, `scanf()`, etc. The corresponding header files must be included using the `#include` directive. The syntax `#include <file_name>` instructs the compiler to search for the file in the standard system directories (as defined by the compiler's include path).

Example: `#include <stdio.h> #include <string.h>`

- **User-defined files:** When a program becomes very large, it is a good practice to divide it into smaller files and include whenever needed. These types of files are user defined files. These files can be included as `#include "filename"`

Example: `#include "sort.h"`

Note: Conventionally, angle brackets `< >` are used for standard header files and double quotes `" "` for user-defined header files. This behavior can be altered by modifying the environment variables or include path settings, allowing user-defined files to be included using angle brackets as well.

Macro Undefinition:

Macros are defined using `#define`, but sometimes it becomes necessary to **remove or cancel** a macro definition. This is done using the `#undef` directive. It is useful when you want to **redefine a macro with a new value** or **avoid name conflicts** in large programs.

Coding Example_12: Undefining a macro

```
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    #undef MAX // Remove the existing macro definition  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Preprocessing stage output:

```
int main() {  
    printf("Before undefining, VALUE = %d\n", 10);  
  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Compilation Error:

```
check.c:6:47: error: 'MAX' undeclared (first use in this function)  
    printf("After undefining, VALUE = %d\n", MAX);  
                                              ^~~~  
check.c:6:47: note: each undeclared identifier is reported only once for each  
function it appears in
```

Coding Example_13: Variable MAX created after Undefining a macro

```
#define MAX 10  
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    #undef MAX // Remove the existing macro definition  
    int MAX = 100;  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

```
C:\Users\Dell>a  
Before undefining, VALUE = 10  
After undefining, VALUE = 100
```

Coding Example_14: Variable creation same as macro, without undefining the macro

```
#define MAX 10  
int main() {  
    printf("Before undefining, VALUE = %d\n", MAX);  
    int MAX = 100;  
    printf("After undefining, VALUE = %d\n", MAX);  
    return 0;  
}
```

Pre-processing stage output:

```
int main() {  
    printf("Before undefining, VALUE = %d\n", 10);  
    int 10 = 100;  
    printf("After undefining, VALUE = %d\n", 10);  
    return 0;  
}
```

Compilation Error:

```
check.c:1:13: error: expected identifier or '(' before numeric constant  
#define MAX 10  
           ^  
check.c:5:10: note: in expansion of macro 'MAX'  
    int MAX = 100;  
        ^~~~
```

Conditional Compilation Directives:

This allows certain parts of the code to be compiled or skipped **based on conditions** and it is useful for writing **platform-independent code**, **debugging**, or **enabling/disabling features**. Can be performed using these directives - `#ifdef`, `#ifndef`, `#if`, `#else`, `#elif`, `#else`, `#endif`. This helps in **Including/excluding features based on build settings**, **Supporting multiple versions or configurations of a program** and, **Simplifying debugging and testing different code paths**.

#if: It allows the compiler to include or exclude parts of the code **based on constant expressions or macro values**.

#elif: It allows **checking another condition if the previous #if or #elif condition was false**. It helps make the code cleaner by avoiding multiple nested `#if-#else` blocks.

#else: Used to define a **default block of code that should be compiled when none of the preceding #if or #elif conditions are true**. It ensures that some code is always compiled, even if no condition matches.

Coding Example_15: Demo of #if #else and #elif

```
#include <stdio.h>
#define VERSION 2 // Change this to 1 and 3.
int main() {
    #if VERSION == 1
        printf("You are using Version 1\n");
    #elif VERSION == 2
        printf("You are using Version 2\n");
    #else
        printf("Unknown version\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {

    printf("You are using Version 2\n");

    return 0;
}
```

Coding Example_16: Demo of #if and #else

```
#include <stdio.h>
int main() {
    #if (2 > 1)
        printf("TRUE\n");
    #else
        printf("False\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {

    printf("TRUE\n");

    return 0;
}
```

#ifdef: “if defined” directive is used to check **whether a macro has been defined** using #define. If the macro is defined, the code inside the #ifdef block is included during preprocessing for compilation. otherwise, it is skipped. It is commonly used to control code compilation for **feature toggles, debugging, and header file protection**.

#ifndef: “if not defined” directive is used to include the statements for compilation if the macro not defined using #define.

#endif: This is used to mark the end of a conditional compilation block that began with #if, #ifdef, or #ifndef. It ensures that the compiler knows where the conditional code segment ends. It is mandatory whenever #if, #ifdef, or #ifndef is used.

Coding Example_17: Demo of #ifdef and #ifndef

```
#define LEVEL 2
int main()
{
    #ifndef MAX //No parentheses for #ifdef
        printf("MAX not defined\n");
    #endif
    #ifdef LEVEL
        printf("LEVEL defined\n");
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main()
{
    printf("MAX not defined\n");

    printf("LEVEL defined\n");
    return 0;
}
```

Coding Example_18: Demo of #ifdef and #ifndef

```
#define LEVEL 2
int main()
{
    #ifndef MAX //No parentheses for #ifdef
        printf("MAX not defined\n");
        #define MAX 10
    #endif
    #ifdef LEVEL
        #ifdef MAX
            printf("LEVEL defined and MAX defined\n");
        #endif
    #endif
    return 0;
}
```

Pre-processing stage output:

```
int main() {
    printf("MAX not defined\n");

    printf("LEVEL defined and MAX defined\n");
}
```

Other Directives

The language supports preprocessor directives that offer more **control over the compilation process**. These directives are typically used for **error handling, file control, custom compiler behavior, and debugging support**. They help in managing the structure of the code, controlling the flow of compilation, enhancing portability across different platforms, and improving debugging and overall maintainability of the program.

#pragma:

A compiler-specific preprocessor directive used to provide additional instructions to the compiler and allows programmers to **enable or disable certain features or optimizations**.

Syntax: `#pragma directive-name [optional-arguments]`

#pragma startup: Runs a function before main()

#pragma exit: Runs a function just before the control returns from main()

#pragma warn: Used to **enable or disable specific compiler warnings**. It helps manage the visibility of certain warnings during compilation.

NOTE: Behavior is unique to Turbo C/C++ and was commonly used in legacy DOS applications.

Coding Example_19:

```
#include <stdio.h>
#include <conio.h>
void start_func() {
    printf("This is called before main()\n");
}
void end_func() {
    printf("This is called after main()\n");
}
#pragma startup start_func
#pragma exit end_func
int main() {
    printf("Inside main function\n");
    return 0;
}
```

`#pragma warn -<letter> // Disable a specific warning`

`#pragma warn +<letter> // Enable a specific warning`

Warning Letter	Meaning
r	Return value is missing in a non-void function
s	Statement is unreachable
p	Parameter is never used
a	Assignment inside a condition (if, while)
h	Function declared but not used
c	Possible loss of precision (conversion)

Coding Example_20:

```
#include <stdio.h>
```

```
#pragma warn -r // Disable "missing return" warning
```

```
int add(int a, int b) {
```

```
    int sum = a + b;
```

```
    // Missing return statement, but no warning
```

```
}
```

```
#pragma warn +r // Re-enable "missing return" warning
```

```
int main() {
```

```
    printf("Result: %d", add(5, 10)); // Undefined behavior
```

```
    return 0;
```

```
}
```

More for you to explore -> `#pragma once`, `#pragma GCC optimize("O3")`, `#pragma GCC poison printf, #line, #error`

Seamless coding with the power of preprocessor directives!